Lecture 19

Graceful Shutdown

Backend Engineering from First Principles

Detailed study notes • lifecycle • signals • connection draining • interview revision

Lecture 19 — Graceful Shutdown

Backend Engineering from First Principles

> *These notes are based on the supplied lecture transcript. Practical clarifications are marked separately where they help prevent overgeneralization.*

1. Why graceful shutdown matters

Imagine a backend is processing a critical payment when a deployment requires the old server process to stop. If the process is terminated abruptly, an in-flight request may be interrupted. Depending on the design, this can lead to failed or ambiguous operations, incomplete work, lost responses, duplicate retries, or difficult recovery.

Graceful shutdown means stopping an application through an orderly sequence rather than terminating it immediately. The application gets an opportunity to finish ongoing work, release resources and then exit.

The lecture's analogy is a restaurant closing:

1. Stop admitting new customers.
2. Let existing customers finish.
3. Close up and clean the premises.

For a backend, the same idea becomes:

1. Stop accepting new work.
2. Finish or safely settle work already in progress.
3. Clean up resources.
4. Exit within a bounded time.

Benefits include:

- Better user experience during deployments and restarts.
- Lower risk of interrupted requests and incomplete workflows.
- Cleaner release of database/network/file resources.
- More predictable process lifecycle and operational behavior.

Important: Graceful shutdown reduces interruption risk; it does not by itself guarantee that a payment or other business operation happens exactly once. Use transactions, idempotency keys and durable workflow design for those guarantees.

2. Process lifecycle management

A backend application runs as a **process** on an operating system. A process has a lifecycle:

- **Start:** the process is created and the application initializes.
- **Run:** the application serves requests and performs work.
- **Stop:** the process receives a shutdown request, performs its shutdown sequence and terminates.

The operating system and process-management infrastructure communicate with processes using mechanisms such as **signals** on Unix-like systems. The application can register handlers so that it responds to selected signals by starting its graceful shutdown procedure.

The lecture focuses on Linux/Unix-like server environments and three signals: `SIGTERM`, `SIGINT` and `SIGKILL`.

3. Important process signals

3.1 `SIGTERM` — polite termination request

`SIGTERM` asks a process to terminate. It is the usual programmatic signal used by deployment systems and process managers.

When the application receives it, it can:

- Stop accepting new connections or work.
- Finish in-flight requests where possible.
- Clean up resources.
- Exit.

The lecture describes it as a gentle nudge: the process gets a chance to handle shutdown itself.

Common sources include deployment systems, orchestrators such as Kubernetes, and process managers such as `systemd` or PM2.

3.2 `SIGINT` — interrupt

`SIGINT` is commonly generated when a user presses **Ctrl+C** in a terminal. It is therefore often encountered during local development.

The lecture recommends handling `SIGINT` and `SIGTERM` through the same graceful-shutdown workflow. The reason is that the intention is the same—stop the application cleanly—even if the signal came from a human or a process manager.

3.3 `SIGKILL` — forceful termination

`SIGKILL` terminates a process immediately. An application cannot catch it, handle it or ignore it. Therefore, it cannot run normal cleanup handlers after receiving `SIGKILL`.

The lecture uses the analogy of pulling the power plug rather than using the normal shutdown procedure.

Signal	Typical meaning/source	Can application handle it?	Graceful cleanup opportunity?
`SIGTERM`	Programmatic termination request	Yes, it can register a handler	Yes
`SIGINT`	Often Ctrl+C in a terminal	Yes, it can register a handler	Yes
`SIGKILL`	Forceful process kill	No	No

Practical note: `SIGKILL` is commonly used as a final fallback when a process has not exited before its shutdown deadline. Do not rely on it for normal shutdown.

4. Connection draining and in-flight requests

An **in-flight request** is a request that has already reached the server and is still being processed when shutdown begins.

The first major step in graceful shutdown is **connection draining**: stop admitting new work and allow existing work to finish.

4.1 Three-step connection-draining process

1. **Stop accepting new connections/requests.**

Prevent new traffic from being assigned to the process.

2. **Allow in-flight work to finish.**

Give active HTTP requests, queries, transactions or other operations time to complete.

3. **Close connections.**

Once work is complete—or the deadline is reached—close the remaining connections and continue cleanup.

The exact implementation depends on the type of backend.

4.2 Different application types

Application type	Graceful shutdown behavior described in the lecture
HTTP backend	Stop accepting new HTTP requests and allow in-flight requests to finish
Database service/application	Stop accepting new queries/transactions, finish current work, then close connections
WebSocket service	Notify clients that the connection is closing, then close sockets

The principle remains the same even though the mechanics differ.

4.3 Coordination with infrastructure

Connection draining is not only an application-code concern. It may require coordination with:

- Load balancers, so new traffic stops being routed to the instance.
- Health checks, so the instance is marked as not ready.
- Service discovery, so other services stop selecting it.
- Deployment/orchestration systems, which manage the transition from old instances to new ones.

The lecture introduces service discovery as the mechanism that helps deployed services find and communicate with each other.

5. Shutdown timeout

A server should not wait forever for an in-flight request. A **shutdown timeout** sets the maximum time allowed for draining and cleanup.

The lecture gives 30 seconds as a common example and also mentions 60 seconds, while emphasizing that the right value depends on the application.

- **Too short:** legitimate requests or long-running operations may be interrupted.
- **Too long:** deployments and restarts become slow, and stuck processes take longer to replace.

Choose the timeout based on:

- Typical and maximum request duration.
- Long-running operations and streaming behavior.
- WebSocket or other persistent connections.
- Queue/worker job duration.
- The deployment platform's termination grace period.
- The service's reliability and operational requirements.

Practical note: The application's internal shutdown deadline should be shorter than the platform's hard termination deadline, leaving time for final cleanup. If a platform sends `SIGKILL` when its grace period expires, application code cannot extend that deadline.

6. Resource cleanup

After stopping new work and allowing active work to finish, the process should release resources it acquired while running.

The lecture lists:

- File handles.
- Network connections.
- Database connections.
- Temporary files.
- Caches and other system resources.
- Background-job processing resources.

Failure to release resources can leave handles or connections open until process exit and can cause leaks or resource exhaustion in long-lived processes. During shutdown, explicit cleanup also helps other components observe orderly closure.

6.1 Database transactions

Before the application exits, transactions it owns should be settled:

- **Commit** a transaction if its intended work has completed successfully.
- **Roll back** a transaction if the work did not complete successfully.

Do not leave transaction outcome ambiguous by abandoning active work. The lecture warns that unsettled transactions can contribute to inconsistent states, deadlocks or other database problems.

6.2 Network and database connections

The lecture describes backend-to-database communication over TCP connections and connection pools. During shutdown:

1. Stop assigning new work to the pool.
2. Allow active queries/transactions to complete or cancel them safely at the deadline.
3. Close the pool and its underlying connections.

6.3 Reverse-order cleanup

A key rule from the lecture:

Release resources in the reverse order in which they were acquired.

Example:

```
Acquire:
1. Redis connection
2. Database connection
3. Start background-job worker

Release:
1. Stop/close background-job worker
2. Close database connection
3. Close Redis connection
```

This is a dependency-aware cleanup strategy: stop components that depend on other resources before closing those underlying resources.

7. Graceful shutdown sequence

A practical high-level sequence based on the lecture:

```
Process is running
|
v
Receive SIGTERM or SIGINT
|
v
Mark instance as shutting down / not ready
|
v
Stop accepting new connections and work
|
v
Drain in-flight HTTP requests and active operations
|
v
Settle database transactions
|
v
Stop background workers and close dependent resources
|
v
Close database / Redis / network / file resources
|
v
Log successful shutdown and exit
```

The exact ordering of individual resources should follow the application's dependency graph and framework requirements.

8. Go backend example from the lecture

The demonstration uses a Go backend. The speaker explicitly says the goal is to understand the lifecycle, not memorize the code.

The example follows this pattern:

1. Register a handler that waits for an interrupt/termination signal.
2. When a supported signal arrives, invoke a graceful-shutdown function.
3. Shut down the HTTP server using the framework/library's shutdown method.
4. Close the database after the server has stopped taking new work and active work has been handled.
5. Stop the background-job processing server, which uses Redis.
6. Log that the server has exited successfully.

The lecture explains that common HTTP frameworks/libraries provide shutdown methods that handle stopping new connections and waiting for existing work, subject to their API and timeout behavior.

Observed demonstration logs

The lecture's local demonstration:

- Starts by connecting to the database.
- Starts the background-job server.
- Starts the HTTP server.
- Receives Ctrl+C (`SIGINT`).
- Begins shutdown and closes the database/server resources in the demonstrated sequence.
- Stops background workers and waits for them to finish.
- Logs successful process exit.

The local server took roughly one second to shut down in the demonstration because there were no in-flight requests. This is an observation from that run, not a universal shutdown duration.

9. Common mistakes to avoid

1. **Abruptly terminating on `SIGTERM`:** misses the opportunity to drain requests and clean up.
2. **Handling only Ctrl+C:** production process managers commonly use `SIGTERM`; handle the signals appropriate to the deployment environment.
3. **Accepting new traffic during shutdown:** makes draining harder and can prolong shutdown.
4. **Waiting forever:** use a bounded shutdown timeout.
5. **Choosing an arbitrary timeout:** base it on real request/job behavior and platform limits.
6. **Closing the database too early:** active handlers may still need it.
7. **Stopping dependencies in the wrong order:** close resources in reverse acquisition/dependency order.
8. **Ignoring background workers:** queued or active jobs also need a defined shutdown policy.

9. **Assuming graceful shutdown guarantees exactly-once payments:** use idempotency, durable state and transaction design for business correctness.
10. **Relying on `SIGKILL` for cleanup:** it cannot be handled by the process.

10. Practical implementation checklist

- ■ Register handlers for the shutdown signals relevant to the runtime (commonly `SIGTERM` and `SIGINT`).
- ■ Make shutdown initiation idempotent so duplicate signals do not start conflicting cleanup sequences.
- ■ Mark the instance as not ready and coordinate with load balancers/service discovery.
- ■ Stop accepting new HTTP connections and new application work.
- ■ Drain active requests within a bounded deadline.
- ■ Define what happens to long-running requests, streams and WebSockets.
- ■ Stop queue consumers from taking new jobs.
- ■ Finish, checkpoint or safely requeue active jobs according to the queue's semantics.
- ■ Commit or roll back active database transactions deliberately.
- ■ Close resources in dependency-aware reverse order.
- ■ Use timeouts/cancellation for each cleanup phase.
- ■ Log shutdown start, phase completion, timeout and final exit.
- ■ Ensure the application's deadline fits inside the platform's termination grace period.
- ■ Test shutdown while requests and background jobs are actively running.

11. Interview questions and answers

Q1. What is graceful shutdown?

It is an orderly process of stopping an application: stop accepting new work, allow existing work to finish or settle safely, release resources and then exit.

Q2. Why is graceful shutdown important during deployment?

The old instance may still be processing requests when a new version becomes ready. Graceful shutdown reduces the chance of interrupting active work and improves deployment reliability.

Q3. What is the difference between `SIGTERM` and `SIGKILL`?

`SIGTERM` is a catchable termination request that gives the application an opportunity to clean up. `SIGKILL` cannot be caught or ignored and stops the process immediately.

Q4. What is `SIGINT` commonly associated with?

A user pressing Ctrl+C in a terminal. It is commonly used during local development.

Q5. What are in-flight requests?

Requests that have already reached the server and are still being processed when shutdown begins.

Q6. What is connection draining?

Stopping new traffic from reaching an instance while allowing existing connections or requests to finish before closing the instance.

Q7. Why is a shutdown timeout needed?

To prevent shutdown from waiting indefinitely for stuck or excessively long operations. It establishes a bounded time for draining and cleanup.

Q8. What resources should be cleaned up?

Examples include file handles, network connections, database pools, temporary files, caches and background workers.

Q9. Why should resources be released in reverse order?

A later-acquired component may depend on an earlier resource. Stopping dependents first avoids closing a resource while another component still needs it.

Q10. How should a backend shut down with a job queue?

Stop accepting new jobs, let active jobs finish or safely requeue/cancel them according to the queue's guarantees, then close worker and broker connections.

Q11. Does graceful shutdown prevent duplicate payment processing?

Not by itself. It reduces abrupt interruption, but payment correctness also requires idempotency keys, durable transaction state and safe retry/reconciliation logic.

Q12. What happens if the process receives `SIGKILL`?

It terminates immediately; the application cannot run its registered cleanup handler.

12. Quick revision sheet

- **Graceful shutdown:** orderly stop, not abrupt termination.
- **Process lifecycle:** start → run → stop.
- `SIGTERM`**:** programmatic termination request; can be handled.
- `SIGINT`**:** commonly Ctrl+C; can be handled.
- `SIGKILL`**:** force kill; cannot be caught or ignored.
- **Connection draining:** stop new work → finish in-flight work → close connections.
- **In-flight request:** already being processed when shutdown starts.
- **Shutdown timeout:** bounded time to finish draining and cleanup.
- **Resource cleanup:** files, sockets, DB pools, temporary resources, workers.
- **Transactions:** commit successful work or roll back incomplete work.
- **Cleanup order:** reverse acquisition/dependency order.
- **Infrastructure coordination:** readiness, load balancer, service discovery and orchestrator.
- **Core idea:** stop new work, settle existing work, clean up, exit.

13. Scope of the supplied lecture

The lecture is an introductory explanation of process lifecycle, Unix signals, connection draining, timeouts, resource cleanup and a Go backend shutdown demonstration. It does not provide a complete framework-specific implementation for every language or deployment platform. The promotional or unrelated material is not part of these notes.